

Markov Decision Processes

Introduction to Planning & Control

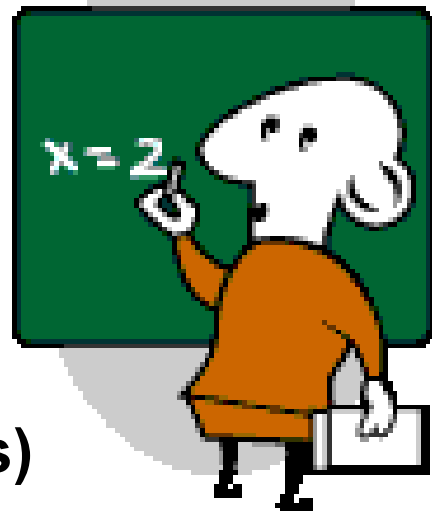
Department of Electrical and Electronics Engineering
Dr. Afşar Saranlı

*Lecture slides heavily use material from the textbook and
Sebastian Thrun, Lecture Slides; <http://www.probablistic-robotics.org/>*



What we will discuss – Part 1

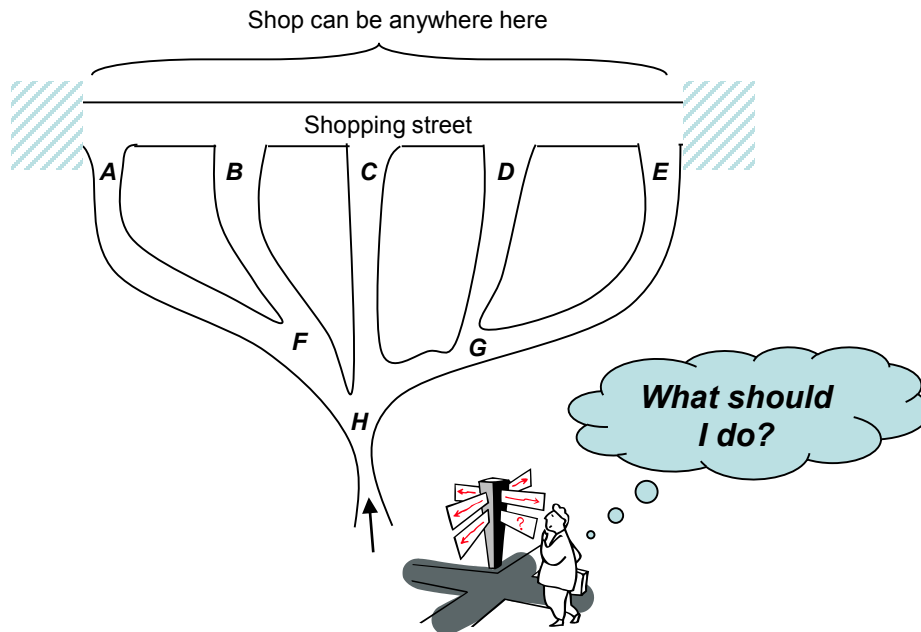
- Discuss our motivations (an example),
- Consider (for the first time) planning & control in the presence of uncertainty,
- Uncertainty in Action selection,
- Introduce Markov Decision Processes (MDPs) and Partially Observable MDPs (POMDPs),
- Focus on motion uncertainty only (MDPs),
- Payoff Functions and Value Iteration Algorithm,
- Examples





Motivation – Small challenge

- We are out shopping, on foot,
- Looking for a particular shop we know to be on a street,
- But: We do not know where exactly is the shop is on the street...
- You can only see the shop when you are next to it!



Where is the non-determinism here?

What would you do if you want to find the shop in minimum time (minimum walking)?

Do you have a strategy?



Motivation - Other Examples

- “A robotic manipulator grasps and assembles parts arriving in random configurations on a conveyer belt. The configurations are not known at the time of arrival but the optimum manipulation requires knowledge of the configuration.”
- *How can we manipulate such pieces?*
- *Is it necessary to sense?*
- *Can we solve manipulation without sensing?*



Motivation - Other Examples

- “An autonomous underwater vehicle is tasked to navigate from Canada to Caspian Sea. It has an Inertial Sensor and a GPS. GPS can only be used on the surface. The vehicle can go through the north pole under the ice (much shorter route - no surfacing), or around the ice in open waters (much longer route)”
- *Which route should you take?*
- *Does the decision depend on the accuracy of inertial sensors?*
- *Other thoughts?*



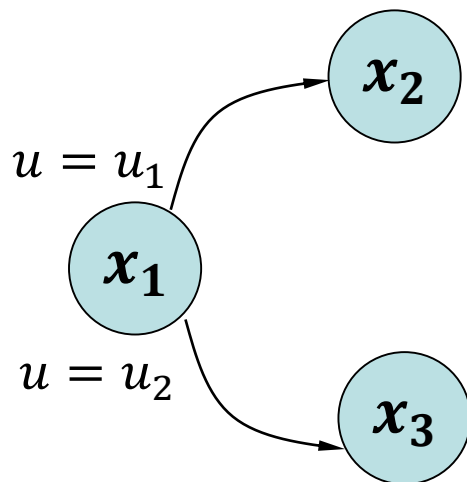
Motivation

- We have exclusively focused on probabilistic “perception” up to now:
 - Localization?,
 - Mapping?,
 - SLAM?
- The ultimate goal of any robot software is to choose the right actions (control) that would accomplish a task.
- Action selection is tied up with the concept of uncertainty.
- Sometimes, actions selected to reduce uncertainty:
“*Information Gathering (Exploration) Tasks*”
- Other times: reducing uncertainty is to achieve some other goal (reaching a goal location reliably and/or in minimum time)

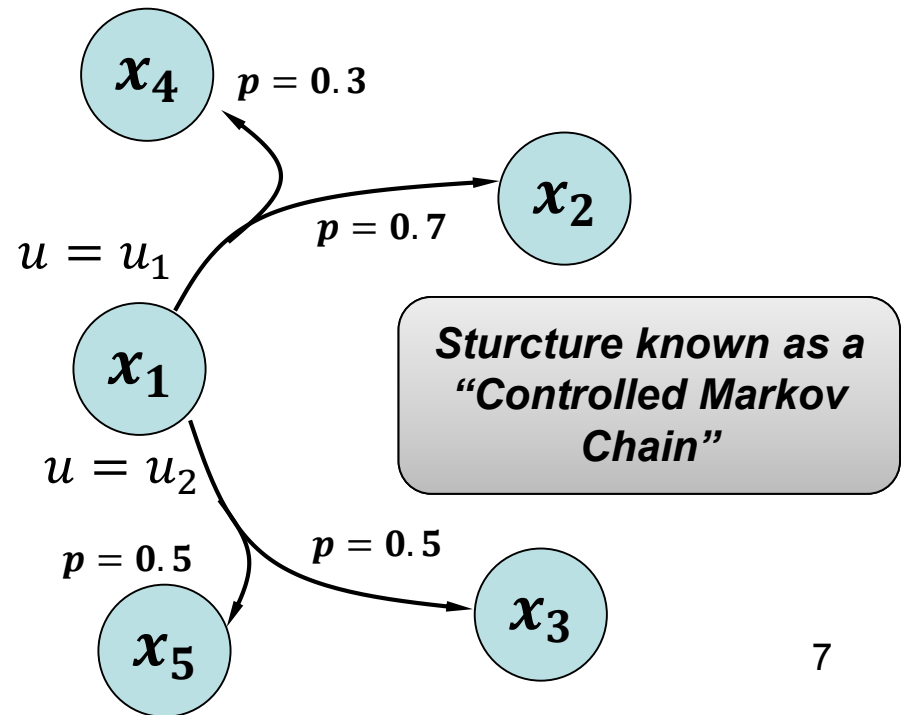


Sources of Uncertainty

- ***Deterministic vs Stochastic Action effects:***
 - The outcomes of robot actions may not be deterministic,
 - Hence: Actions cause uncertainty in state space,
 - The environment state may be stochastic,
 - Robot needs sensing to determine subsequent actions,
 - A discrete-state example:



vs.





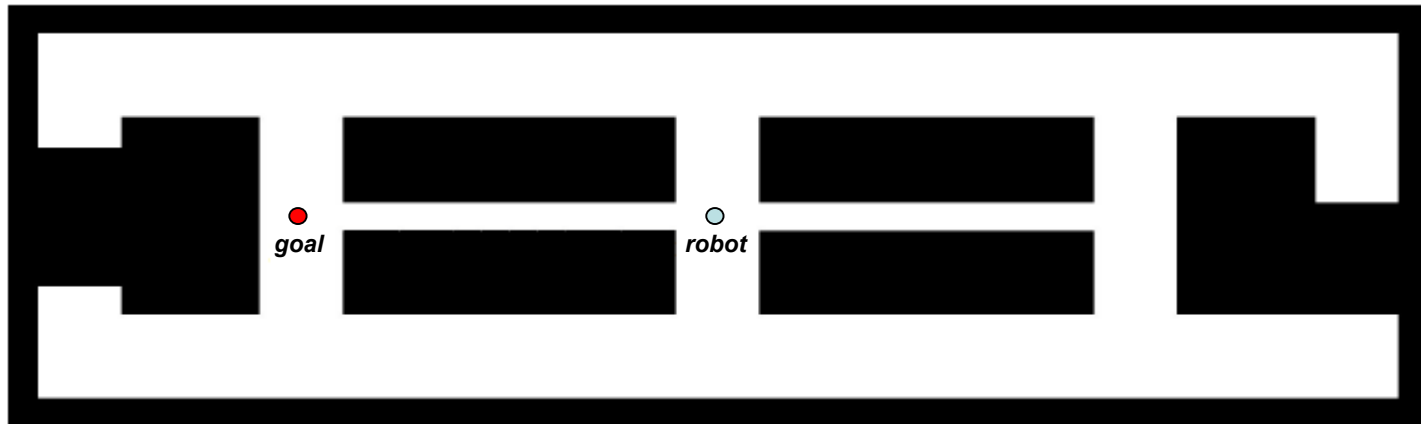
Sources of Uncertainty

- ***Fully Observable vs Partially Observable state spaces:***
 - Classical robotics assume that sensors can measure the full state of the environment!!
 - In all realistic real-world robotic problems, sensor limitations are key factors. (we can only observe sensor outputs)
- **Robots should be able to deal with current uncertainty**
(you cannot simply pre-plan but: need to react to unexpected changes in the state)
- **as well as deal with anticipated uncertainty**
(you have to consider which actions would reduce or increase uncertainty and act to reduce future uncertainty)
- For our discussion: ***planning = control = action selection***



Toy Environment

- Consider the following toy environment,
- Highly symmetric with corridors except for the far left and far right end points,

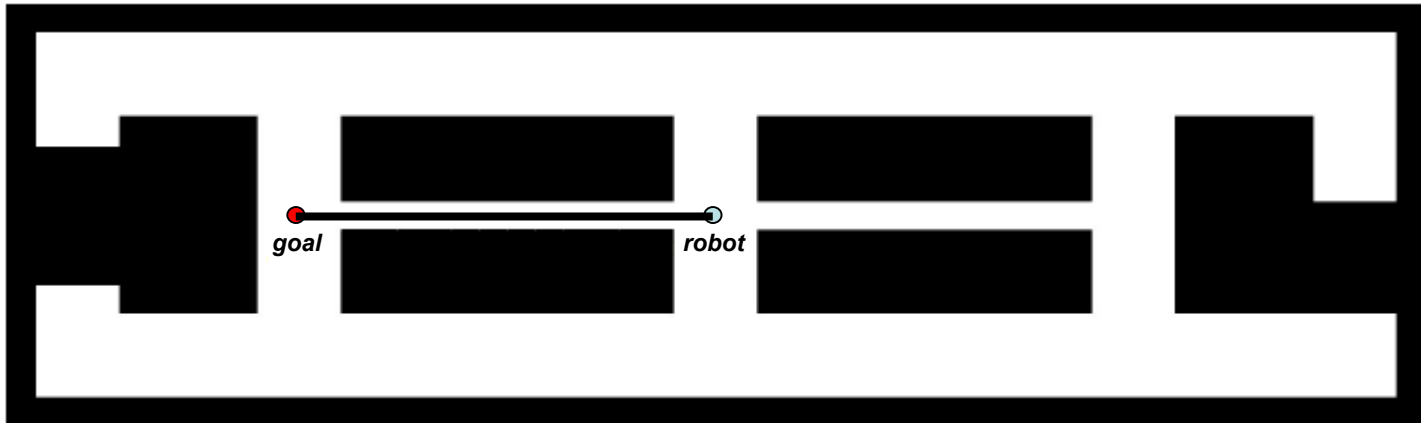


- Multiple paths to the goal,
- How can the robot select actions in order to reach the goal?



Deterministic – Fully Observable

- The robot state, the map (map and goal location in it) is known in full.

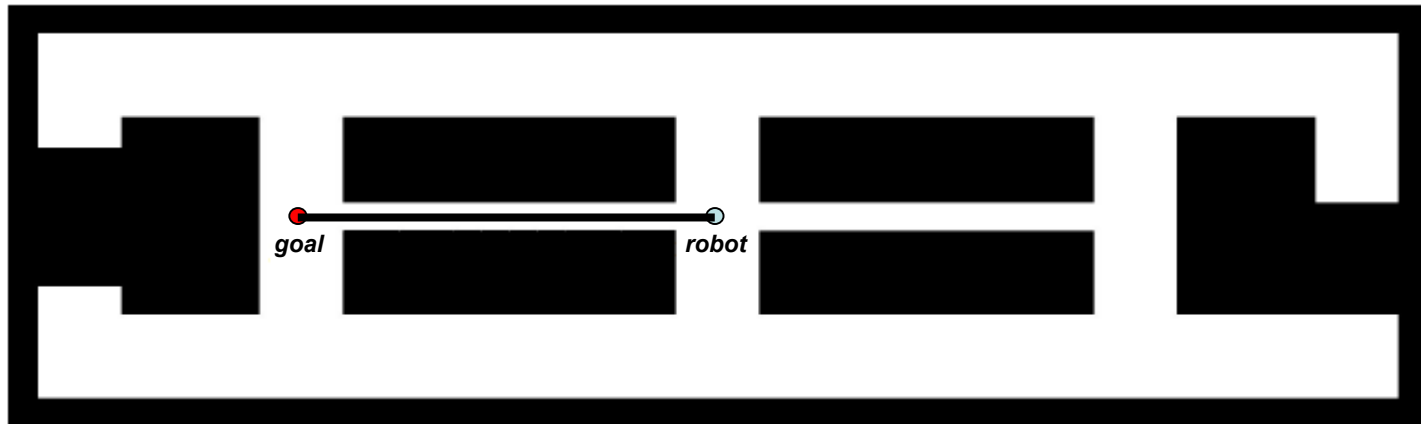


- The robot can plan off-line a deterministic path from source to destination. Then execute it.



Non-Deterministic – Fully Observable

- What if the actions of the robot can have non-deterministic outcomes (state uncertainty)?
- What is the problem with the previous solution?

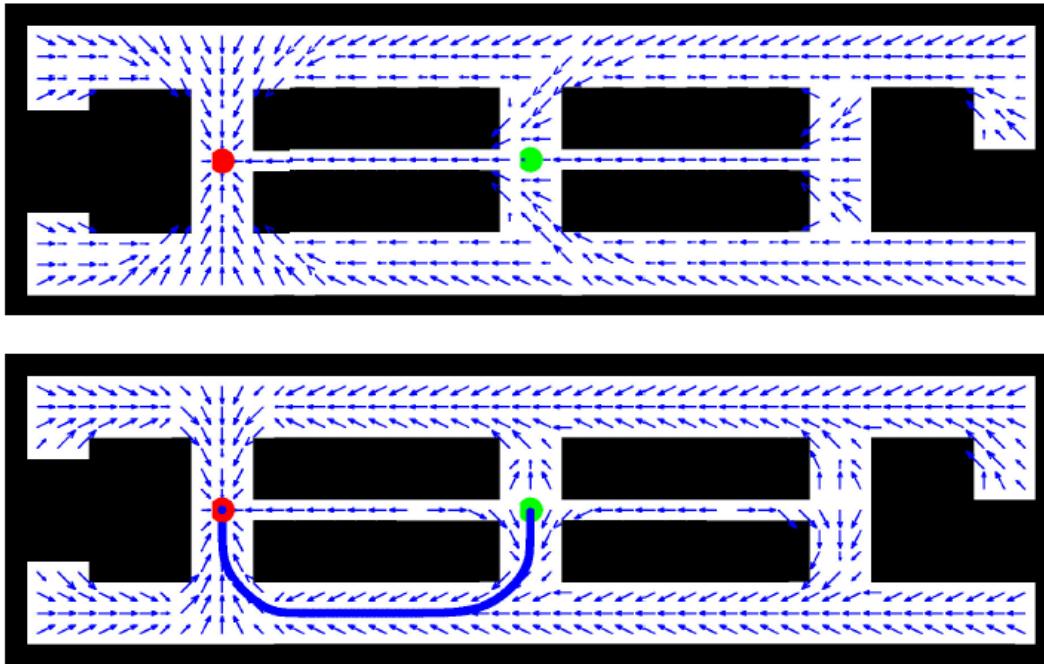


- The robot (due to *its actions* or some *other disturbance*) can find itself in some arbitrary state!!
- It should have action selections for any such state!!



Non-Deterministic – Fully Observable

- Needed: *a mapping from all possible states to robot actions*,
- It is known as a *“control policy”* (in this case a *“state-feedback”*)



*“Control Policy”
“Universal Plans”
“Navigation Functions”*

- An optimal “control policy” should be determined for the task that the robot needs to complete - Called the Markov Decision Process
- This depends on the probabilistic model of robot actions and the map! (later for “partially observable” case the model of perception)

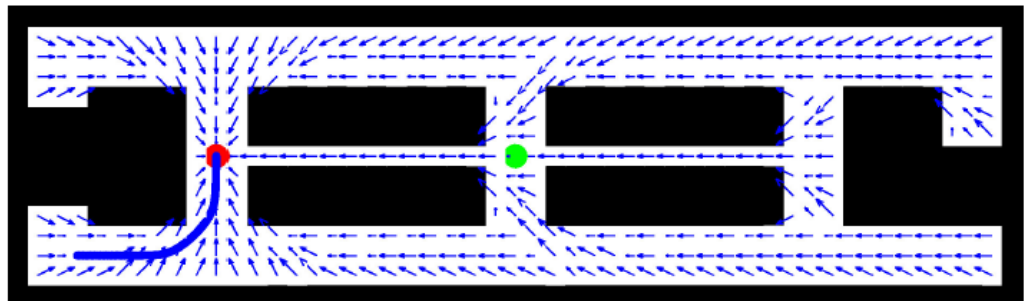
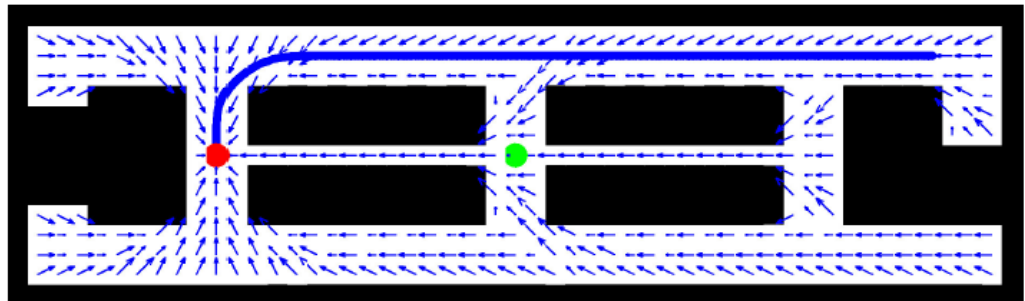
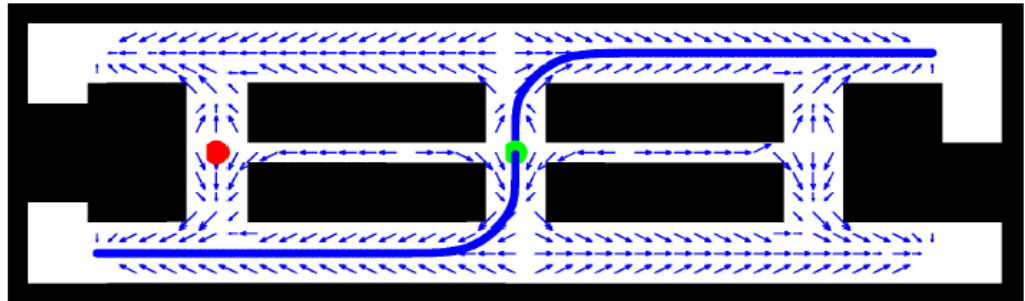


Non-Deterministic - Partially Observable

- The robot state cannot be exactly measured but characterized by a probabilistic model

Example of POMDP:

To maximize the chance of reaching the goal, the planner first executes actions to reduce uncertainty in robot state (active localization) then actions to reach the goal



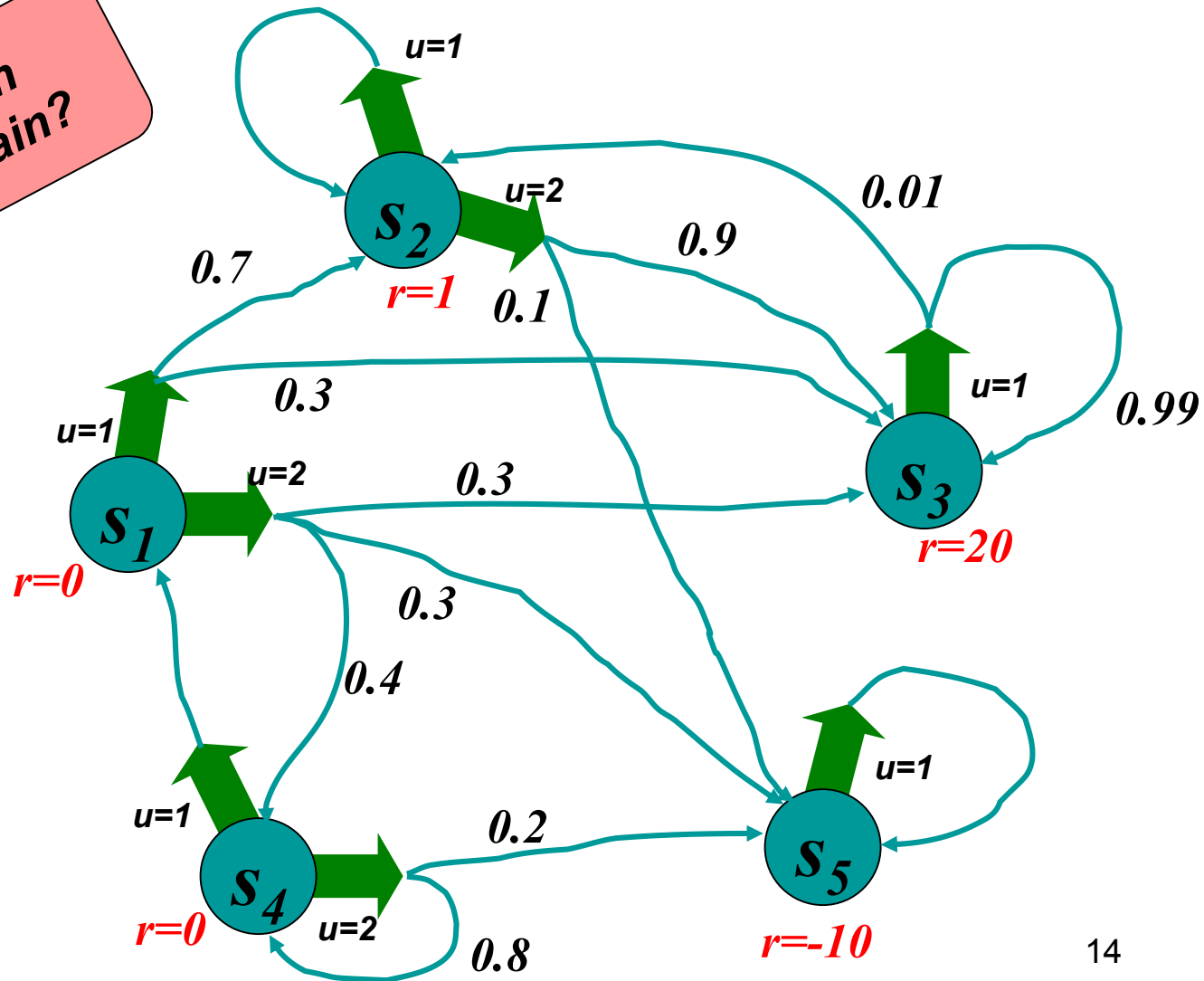


The Markov Decision Process (MDP)

Any thoughts on this Markov Chain?

Given a fully observable controlled Markov Chain and a "Reward/Payoff Function r "

What is the control policy u with respect to a performance criteria?





The Markov Decision Process (MDP)

- **Given:**
- States x
- Actions u
- Transition probabilities $p(x'|u, x)$
- Reward / payoff function $r(x, u)$

- **Wanted:**
- Policy $\pi(x)$ that maximizes the performance criteria “future expected reward”



Goals, Costs and Payoff

- Action selection is driven by “Goals” (a specific configuration or other conditions).
- “Reaching the goal” is a source of *reward*,
- While trying to reach the goal, system may be penalized, (consuming energy, time, other resources),
- Both concepts can be conveniently combined in a single “*Payoff Function*”, Example:

$$r(x, u) = \begin{cases} +100 & \text{if } u \text{ leads to a goal configuration} \\ -1 & \text{otherwise} \end{cases}$$



Rewards and Policy

- Policy (general case):

$$\pi: z_{1:t-1}, u_{1:t-1} \rightarrow u_t$$

- Policy (fully observable case):

$$\pi: x_t \rightarrow u_t$$

- Expected cumulative payoff.**

$$R_T = E \left[\sum_{\tau=1}^T \gamma^{\tau} r_{t+\tau} \right]$$

“Planning Horizon”

“Discount Factor” < 1

- $T = 1$: “greedy” policy: *Grab the immediate reward!!*
- $T > 1$: finite horizon case, typically no discount
- $T \rightarrow \infty$: infinite-horizon case, finite reward if discount $\gamma < 1$.



Finding the Optimal Policy

- Expected cumulative payoff of given policy (*for fully observable case*):

$$R_T^\pi(x_t) = E \left[\sum_{\tau=1}^T \gamma^\tau r_{t+\tau} \mid u_{t+\tau} = \pi(x_t) \right]$$

- Optimal policy:

$$\pi^* = \operatorname{argmax}_{\pi} R_T^\pi(x_t)$$

- 1-step optimal policy:

$$\pi_1(x) = \operatorname{argmax}_u r(x, u)$$

- “Value function” of 1-step optimal policy:

$$V_1(x) = \gamma \max_u r(x, u)$$



2-Step (Horizon 2) Optimal Policy

- **Optimal policy:**

$$\pi_2(x) = \operatorname{argmax}_u \left[r(x, u) + \int V_1(x') p(x' | u, x) dx' \right]$$

- **Value function:**

$$V_2(x) = \gamma \max_u \left[r(x, u) + \int V_1(x') p(x' | u, x) dx' \right]$$



2-Step Optimal Illustration

$$\pi_2(x) = \operatorname{argmax}_u \left[r(x, u) + \int V_1(x') p(x' | u, x) dx' \right]$$

$$V_2(x) = \gamma \max_u \left[r(x, u) + \int V_1(x') p(x' | u, x) dx' \right]$$

From x we can go to $x' = x'_1, x'_2$ or x'_3 with some probabilities. The “reward” of such transitions are given by the 1 step value functions. Take the expectation over those to obtain reward of u .

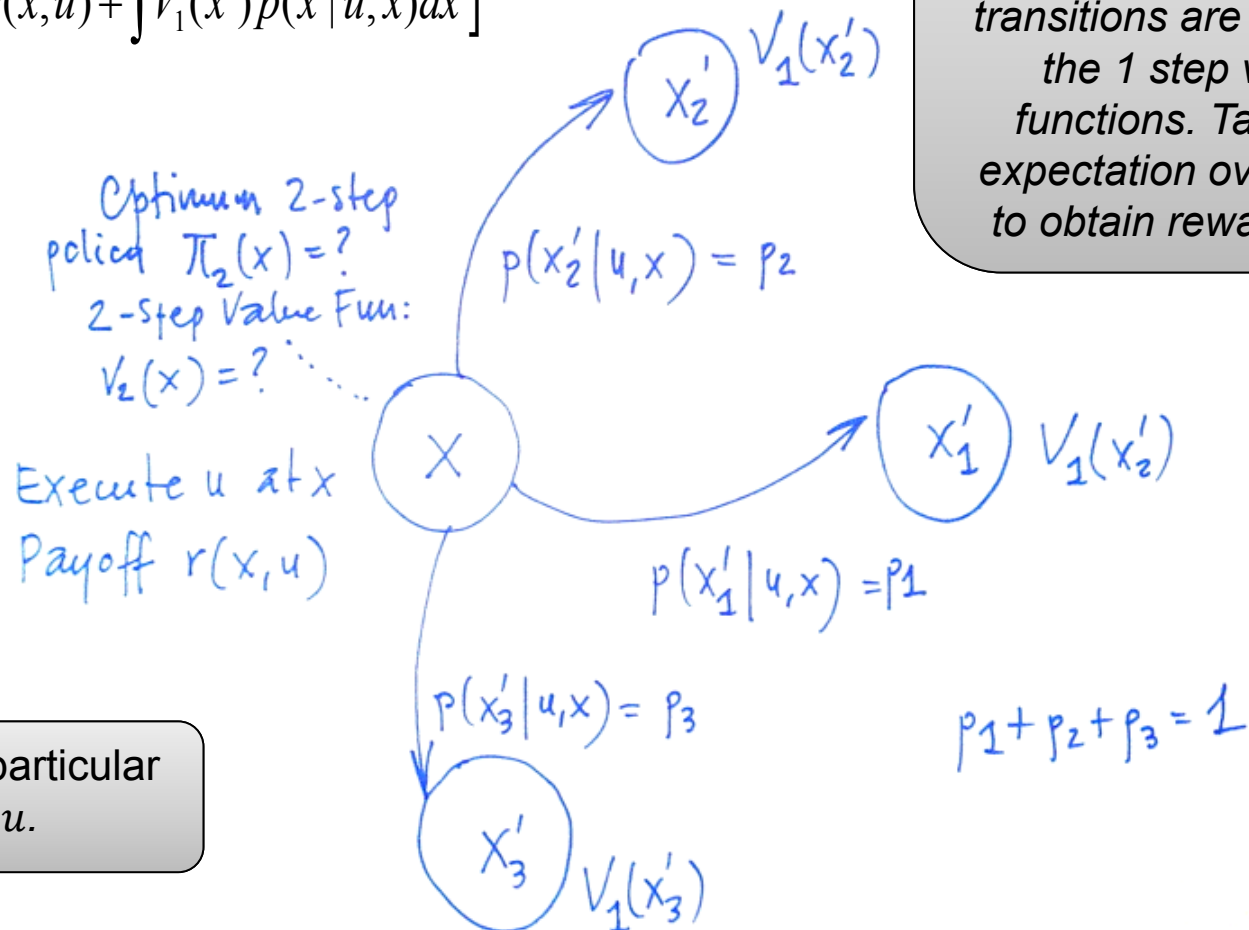


Illustration for a particular command u .



T -Step (Horizon T) Optimal Policy

- **Optimal policy:**

$$\pi_T(x) = \operatorname{argmax}_u \left[r(x, u) + \int V_{T-1}(x') p(x' | u, x) dx' \right]$$

- **Value function:**

$$V_T(x) = \gamma \max_u \left[r(x, u) + \int V_{T-1}(x') p(x' | u, x) dx' \right]$$



Infinite Horizon

- **Optimal value function reaches an equilibrium**

$$V_{\infty}(x) = \gamma \max_u \left[r(x, u) + \int V_{\infty}(x') p(x' | u, x) dx' \right]$$

- **$V_{\infty}(x)$ satisfies the “Bellman Equation”**
- **Any policy whose value function is a fixed point of the “Bellman Equation” (solves it) is an optimal policy**
- **Necessary and sufficient condition**



How to compute the Value Function?

- The algorithm called “Value Iteration” aims to solve the Bellman Equation “numerically” by successive approximations,
- Value Iteration starts from a minimum possible immediate payoff for all states:

$$\hat{V}(x) \leftarrow r_{\min}$$

- Then successively computes the value function for increasing horizons:

$$\hat{V}(x) \leftarrow \gamma \max_u \left[r(x, u) + \int \hat{V}(x') p(x' | u, x) dx' \right]$$

- Until convergence.



Optimal Policy from Value Function

- Once the value function converges and is determined (stabilized) for all states, the optimal policy is given by:

$$\pi(x) = \operatorname{argmax}_u \left[r(x, u) + \int \hat{V}(x') p(x' | u, x) dx' \right]$$



Value Iteration Algorithm - Continuous

```
1:  Algorithm MDP_value_iteration( ):
2:      for all  $x$  do
3:           $\hat{V}(x) = r_{\min}$ 
4:      endfor
5:      repeat until convergence
6:          for all  $x$ 
7:               $\hat{V}(x) = \gamma \max_u \left[ r(x, u) + \int \hat{V}(x') p(x' | u, x) dx' \right]$ 
8:          endfor
9:      endrepeat
10:     return  $\hat{V}$ 
```



Value Iteration Algorithm - Discrete

```
1:  Algorithm MDP_discrete_value_iteration():
2:      for  $i = 1$  to  $N$  do
3:           $\hat{V}(x_i) = r_{\min}$ 
4:      endfor
5:      repeat until convergence
6:          for  $i = 1$  to  $N$  do
7:              
$$\hat{V}(x_i) = \gamma \max_u \left[ r(x_i, u) + \sum_{j=1}^N \hat{V}(x_j) p(x_j \mid u, x_i) \right]$$

8:          endfor
9:      endrepeat
10:     return  $\hat{V}$ 
```



Getting the Optimal Policy

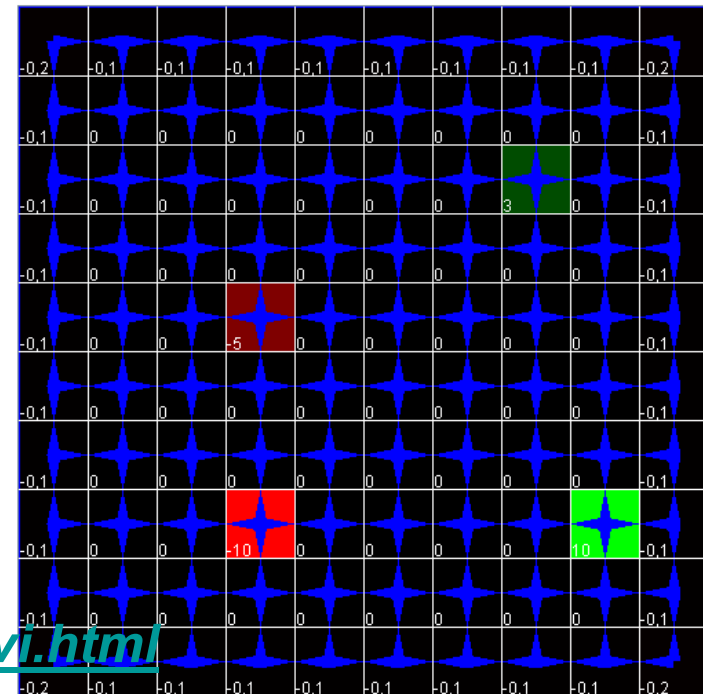
1: **Algorithm** $\text{policy_MDP}(x, \hat{V})$:

2: $\text{return } \underset{u}{\operatorname{argmax}} \left[r(x, u) + \sum_{j=1}^N \hat{V}(x_j) p(x_j \mid u, x_i) \right]$



Illustration Example

- Simple grid world,
- 4 actions available: Up, Down, Left, Right
- Probabilistic motion model:
(0.7 chance of going correct, 0.1 chance a random motion along any other direction)
- Bumping walls:
-1 reward, no motion,
- 4 squares with actual rewards, *on exit*,





Value Iteration vs Policy Iteration

- **Often the optimal policy has been reached long before the value function has converged.**
- **Policy iteration calculates a new policy based on the current value function and then calculates a new value function based on this policy.**
- **This process often converges faster to the optimal policy.**

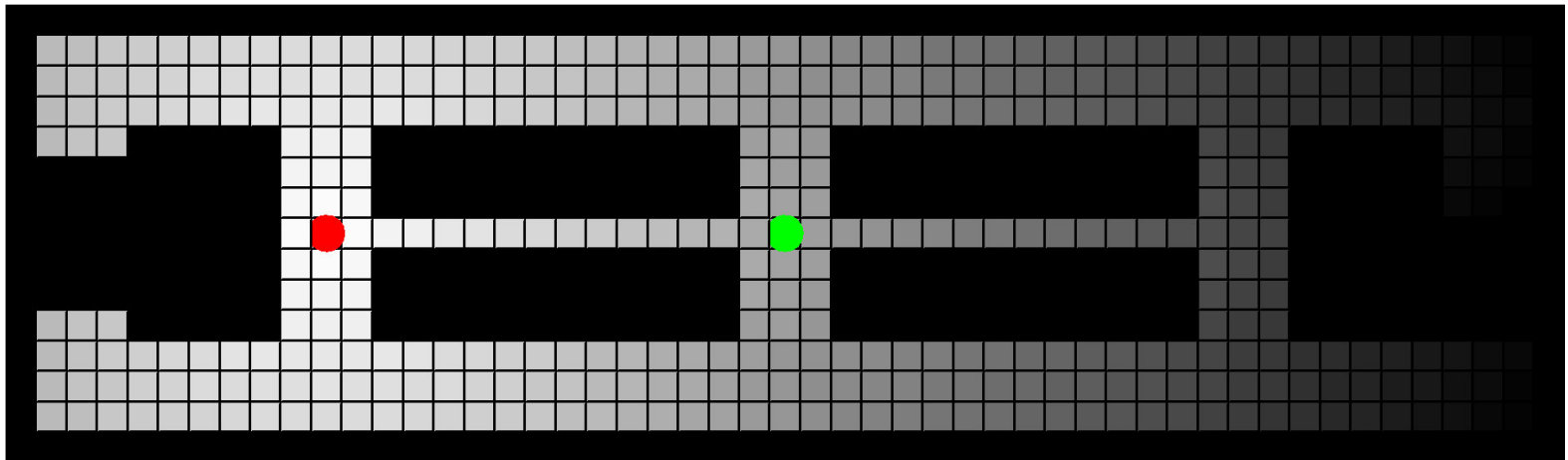


Applications to Robot Motion Control

- **Robot motion planning/control,**
- **The fully-observable case is applicable to a small number of problems where low dimensional state is fully observable,**
- **Example: 2D Robot Motion Planning**
- **We need other considerations:**
 - **Discretize the problem: Grid approximation for the map and robot state,**
 - **Consider the “configuration space transform”, i.e., transform the robot to a point and “expand the obstacles”,**
 - **Define the “state” as the mean of the posterior**



Discrete Map and State



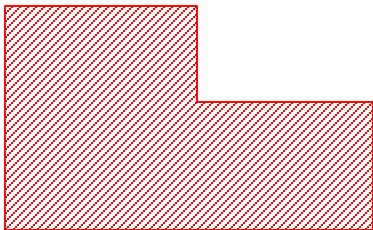


C-Space Transform



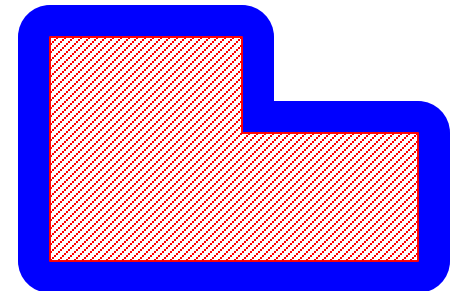
2-D World
2 DOFs

Where can I
move this **robot**
in the vicinity of
this **obstacle**?



...is
equivalent
to...

Where can I
move this **point**
in the vicinity of
this **expanded**
obstacle?



Note: It becomes more interesting if robot has non-symmetric shape and can rotate



C-Space Transform Examples



2-D World
2 DOFs

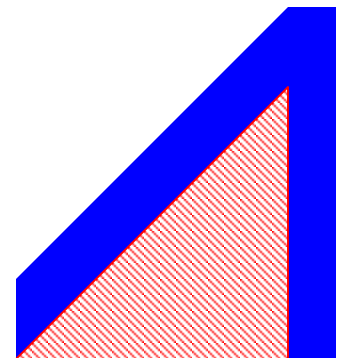
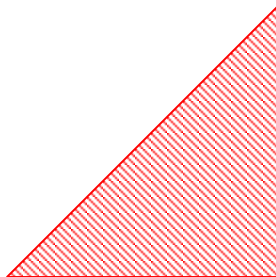
Where can I
move this **robot**
in the vicinity of
this **obstacle**?

...is
equivalent
to...

Where can I
move this **point**
in the vicinity of
this **expanded**
obstacle?

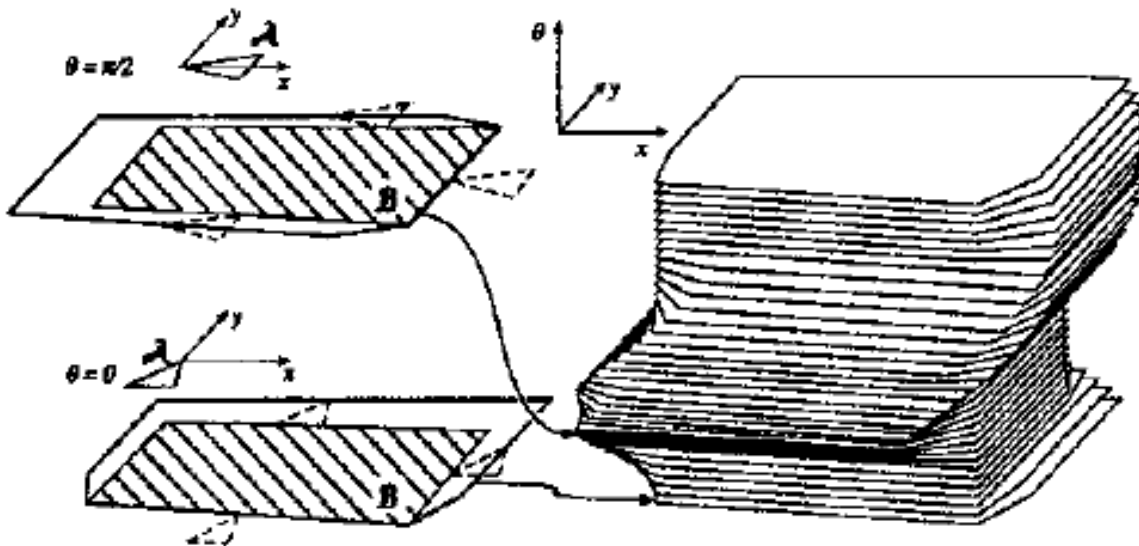
Question: How about a with
rotation?

Assuming you're
not allowed to
rotate

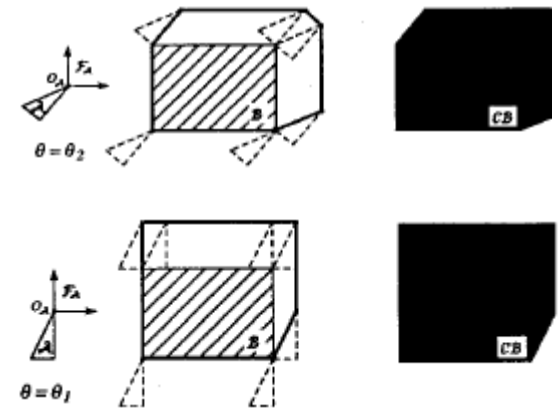




C-Space Transform Examples



2-D World
3 DOFs



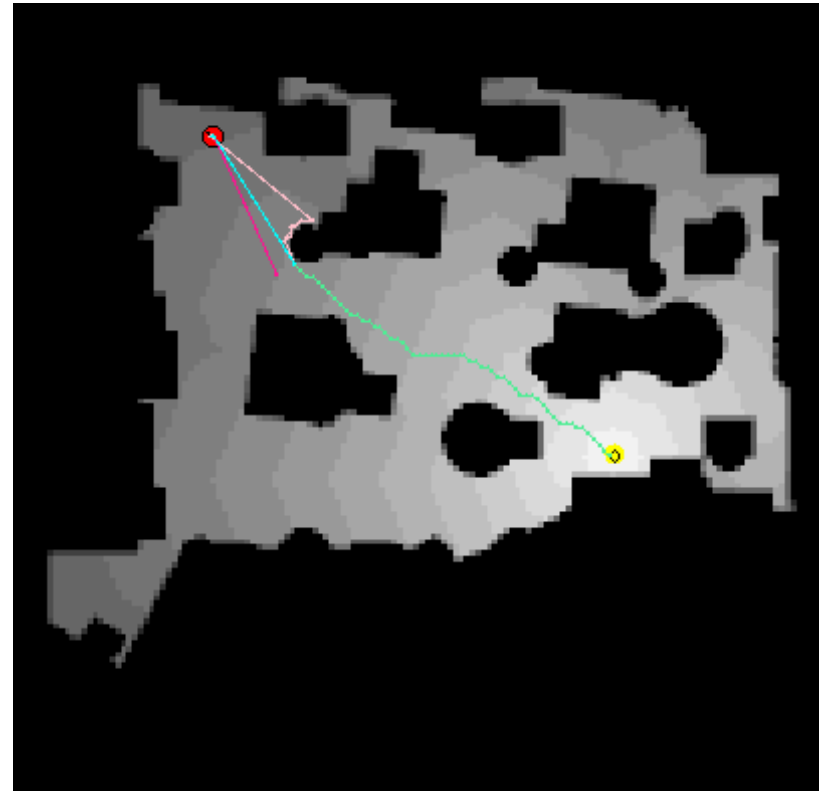
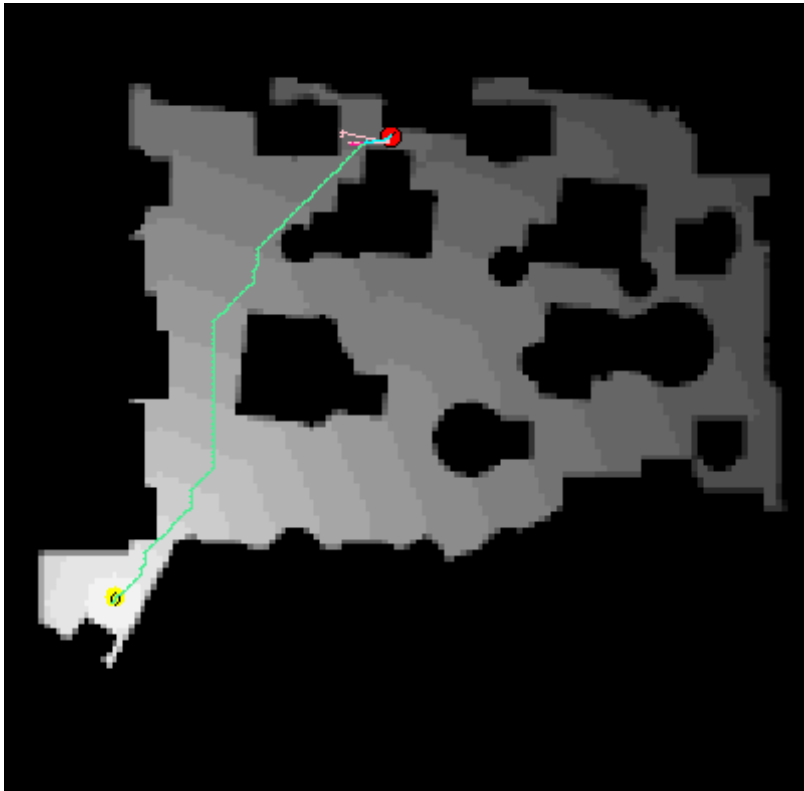
Examples from J.C. Latombe "Robot Motion Planning" (Kluwer 1990)

- We've turned the problem from **"Twist and turn this 2-D polygon past this other 2-D polygon"** into **"Find a path for this point in 3-D space past this weird 3-D obstacle"**.
 - Why's this transform useful?
- Because we can plan paths for points instead of polyhedra/polygons



Value Iteration for Motion Planning

- Mobile robot example

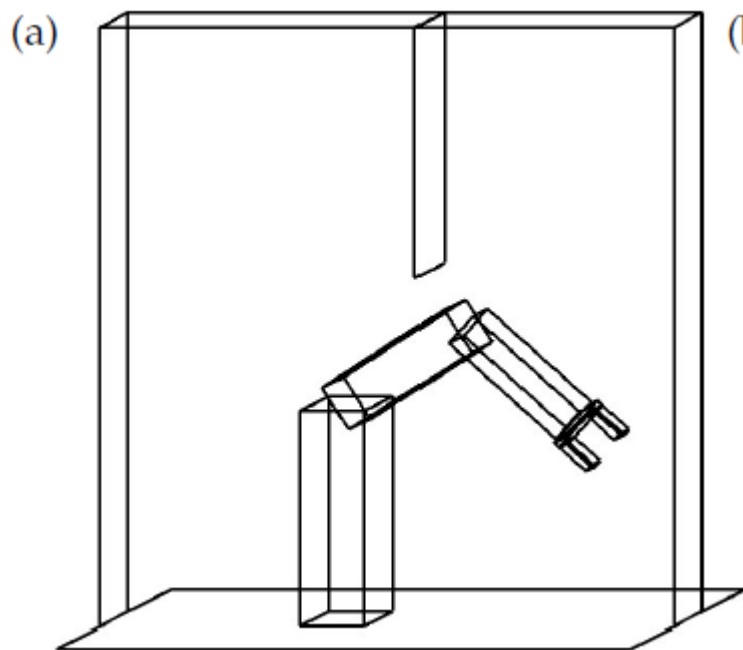


- Value function is indicated by gray shades
- Black areas are “extended obstacles” in Configuration Space (C-Space)
- A Greedy Optimal Policy (Select the cell with the largest Value Function – Hill Climbing)

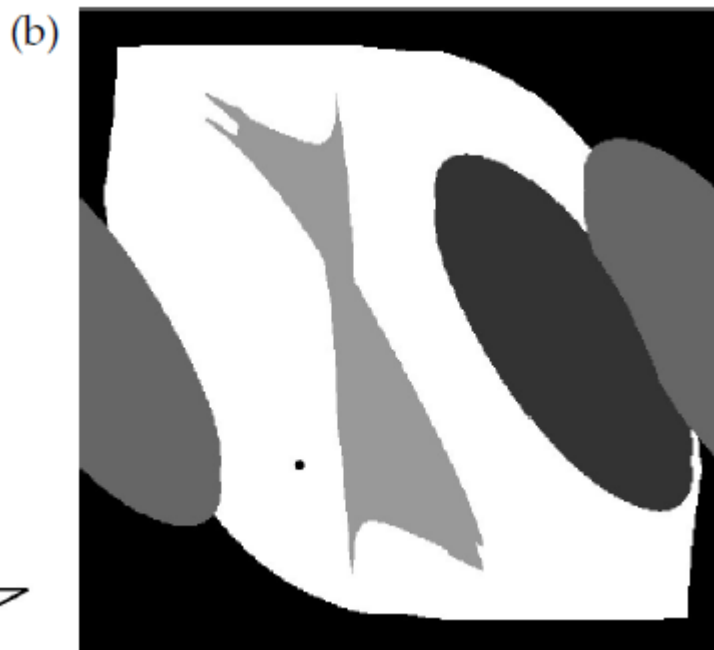


Value Iteration for Motion Planning

- Manipulator Example



“Work space”



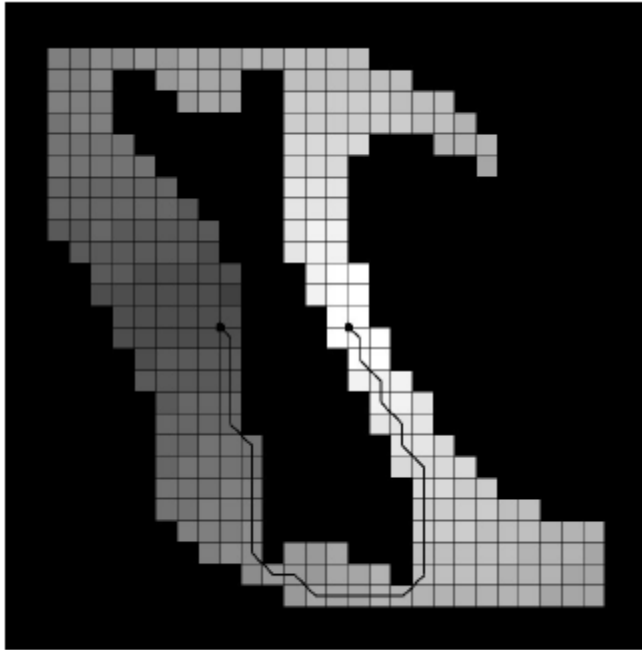
“Configuration Space”



Value Iteration for Motion Planning

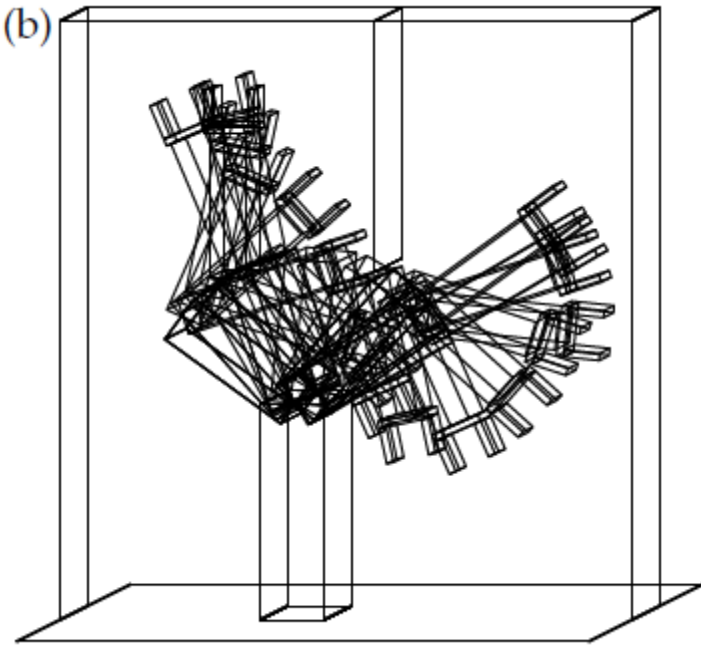
- Manipulator Example – Value Iteration, Coarse Grid

(a)



“Configuration Space”

(b)



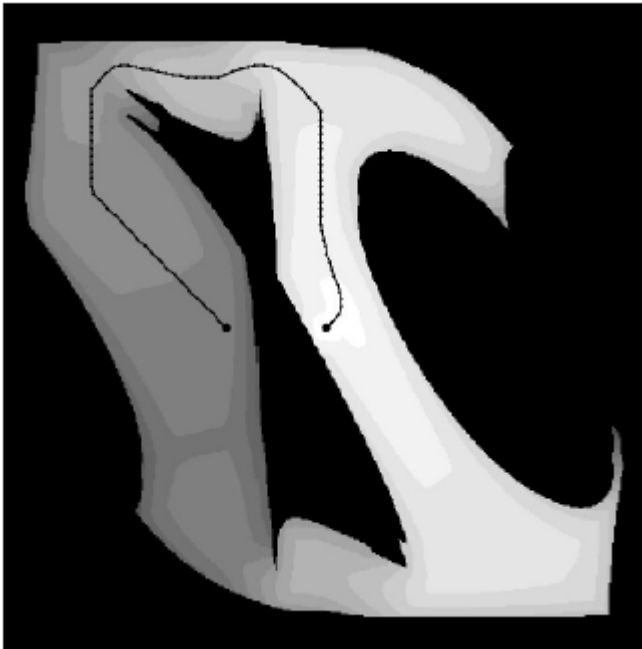
“Work space”



Value Iteration for Motion Planning

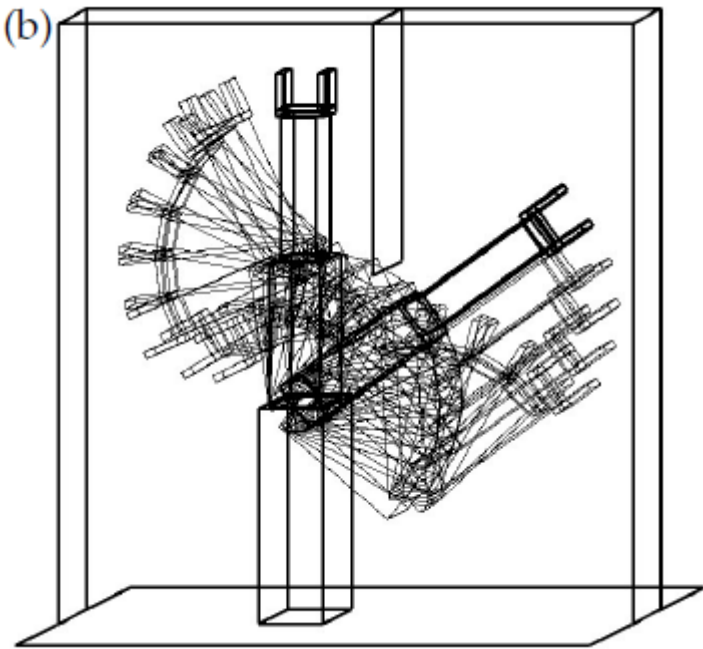
- Manipulator Example – Value Iteration, Fine Grid

(a)



“Configuration Space”

(b)



“Work space”



Summary

- **Class's turn to summarize!**



Up Next: